INFORME

Refactorización con Principios SOLID – Sistema de Gestión de Clínica Odontológica

Proceso de Desarrollo de Software – UADE 2026

Integrantes: Daher, Ezequiel · Quesada, Bautista · Claros, Alex

1. Introducción

El presente informe documenta la revisión y refactorización del sistema de gestión para una clínica odontológica, tomando como criterio de análisis los cinco principios SOLID de diseño orientado a objetos. El sistema, desarrollado en Java con interfaz gráfica Swing y persistencia en archivos de texto, administra pacientes, odontólogos, secretarias y turnos.

El trabajo partió de la versión resultante de las dos iteraciones anteriores, en las que se corrigieron los code smells y se reasignaron responsabilidades según los patrones GRASP. El software era funcional y su diseño ya respetaba buena parte de los criterios buscados: los servicios dependían de interfaces, la creación de colaboradores estaba centralizada en DependenciasClinica y el módulo de Turno tenía sus responsabilidades repartidas en clases específicas. Sin embargo, una revisión orientada a SOLID mostró que todavía quedaban puntos donde el diseño no admitía extensiones sin modificar código existente, y donde las abstracciones disponibles no se correspondían con lo que cada cliente necesitaba.

El diagnóstico central fue el siguiente: las iteraciones anteriores habían resuelto qué clase debía hacer cada cosa, pero no siempre cómo debían relacionarse esas clases entre sí. Las familias que el negocio puede ampliar, como las coberturas y las especialidades, estaban modeladas como enums cerrados. Las interfaces de repositorio se entregaban completas a clientes que solo usaban una o dos operaciones. La capa de persistencia, por su parte, seguía tipada contra la clase ArchivoTexto y contra los repositorios concretos, de modo que la inversión de dependencias lograda en los servicios se interrumpía al llegar a ella. En la vista, además, los paneles de Paciente, Odontólogo y Secretaria no habían recibido la división en componentes que TurnoPanel ya tenía.

Al igual que en la iteración anterior, la estrategia adoptada no fue rediseñar el sistema, sino extender a las partes rezagadas soluciones que ya funcionaban en otras zonas del proyecto. La separación de TurnoPanel en formulario, tabla y filtros sirvió de modelo para los demás paneles, y el uso de interfaces en los servicios sirvió de modelo para la persistencia.

Se relevaron nueve problemas de diseño, identificados como P-01 a P-09. Cuando un mismo problema afecta a más de un principio, se lo registra una sola vez indicando todos los principios involucrados. Los nueve fueron corregidos y verificados. El análisis del Principio de Sustitución de Liskov no arrojó violaciones, y su resultado se detalla en la sección 5.

2. Metodología de Trabajo

El procedimiento aplicado a cada hallazgo constó de cuatro pasos. Primero se identificó la clase o el contrato que originaba el problema, junto con el motivo de cambio, la extensión o la dependencia que quedaba comprometida. Luego se determinó qué principio describía la situación, aplicando Responsabilidad Única cuando una clase reunía motivos de cambio independientes, Abierto/Cerrado cuando una extensión obligaba a editar código existente, Segregación de Interfaces cuando un cliente dependía de operaciones que no usaba, e Inversión de Dependencias cuando un módulo de alto nivel dependía de una implementación concreta. En tercer lugar se evaluó la alternativa elegida, verificando que cada abstracción nueva respondiera a un punto de variación real y no se agregara solo por las dudas, el mismo criterio que en la primera iteración llevó a rechazar la complejidad artificial. Finalmente se implementó el cambio, se recompiló el sistema completo y se revisó el código resultante para confirmar que el problema no persistiera en otras clases.

3. Resumen de Hallazgos

La siguiente tabla resume los nueve problemas detectados, los principios que sustentan cada corrección y las clases donde puede verificarse el resultado.

ID	Problema detectado	Principios aplicados	Dónde verlo
P-01	Los paneles de actores concentraban responsabilidades	Responsabilidad Única	Paneles de Paciente, Odontólogo y Secretaria
P-02	Filtros de turnos centralizados y rígidos	Responsabilidad Única · Abierto/Cerrado	TurnoFiltroDialogos y TurnoFiltrosPanel
P-03	La gestión de turnos mezclaba consultas, comandos y facturación	Responsabilidad Única · Segregación de Interfaces	TurnoService y FacturacionTurnoService
P-04	MainFrame mezclaba composición visual con cierre y guardado	Responsabilidad Única	MainFrame y Manejador CierreAplicacion
P-05	Agregar una especialidad obligaba a modificar la fábrica	Abierto/Cerrado	OdontologoFactory y CreadorOdontologo
P-06	Las coberturas estaban cerradas en un enum	Abierto/Cerrado	CoberturaPaciente y CatalogoCoberturas

ID	Problema detectado	Principios aplicados	Dónde verlo
P-07	Interfaces de repositorio demasiado amplias	Segregación de Interfaces · Inversión de Dependencias	Contratos de repository y colaboradores de Turno
P-08	La persistencia dependía de ArchivoTexto	Inversión de Dependencias	AlmacenamientoLineas y clases Persistencia
P-09	La persistencia dependía de repositorios concretos	Inversión de Dependencias	PersistenciaServicio y DependenciasClinica

4. Correcciones Implementadas

P-01 – Los paneles de actores concentraban responsabilidades

Situación previa. PacientePanel, OdontologoPanel y SecretariaPanel reunían en una sola clase la construcción de la tabla, la construcción y limpieza del formulario, la conversión de los campos a objetos de transferencia, la administración de la selección, la ejecución de altas, modificaciones, bajas y búsquedas, y la presentación de mensajes. Un cambio en las columnas, en los campos o en el flujo de interacción obligaba a modificar la misma clase. TurnoPanel ya había sido dividido en la iteración anterior, pero los otros tres paneles conservaban esa concentración.

Principios aplicados. Por Responsabilidad Única, la representación tabular, la captura y traducción de datos y la coordinación de las acciones del usuario son motivos de cambio independientes, y cada uno debe quedar en una clase distinta. Un cambio en las columnas de la tabla no debería obligar a revisar el código que arma el formulario.

Cambio aplicado. Se crearon PacienteFormularioPanel, PacienteTablaPanel, OdontologoFormularioPanel, OdontologoTablaPanel, SecretariaFormularioPanel y SecretariaTablaPanel. Los componentes de tabla definen las columnas y notifican la selección; los formularios contienen los campos y los convierten en objetos de transferencia; y cada panel principal quedó a cargo de coordinar los eventos, comunicarse con su controlador y presentar los mensajes.

Dónde verlo. Los tres paneles principales y sus seis componentes. PacientePanel pasó de 283 a 133 líneas, OdontologoPanel de 234 a 132 y SecretariaPanel de 209 a 131. Los cuatro módulos de la vista siguen ahora la misma estructura.

P-02 – Los filtros de turnos estaban centralizados y eran rígidos

Situación previa. TurnoPanel abría los diálogos de filtrado, interpretaba los identificadores y las fechas ingresadas y manejaba los errores de formato, además de coordinar el resto de la pantalla. Los métodos filtrarPorPaciente, filtrarPorOdontologo, filtrarPorSecretaria, filtrarPorFechas, filtrarPorEstado y solicitarId permanecían en esa clase. TurnoFiltrosPanel, por su parte, recibía seis Runnable mediante un constructor fijo, uno por cada botón. Agregar un filtro obligaba a modificar ambas clases y a ampliar ese constructor.

Principios aplicados. Por Responsabilidad Única, capturar y convertir criterios de búsqueda es una tarea distinta de coordinar altas, modificaciones, bajas y la actualización de la tabla. Por Abierto/Cerrado, el conjunto de filtros no podía ampliarse agregando una acción nueva de forma independiente, sino que exigía editar el panel de filtros, su constructor y TurnoPanel.

Cambio aplicado. Se creó AccionFiltroTurno, que asocia el texto de un botón con la acción que ejecuta, y TurnoFiltroDialogos, que concentra los diálogos, la conversión de los criterios y la ejecución de las consultas. TurnoFiltrosPanel recibe ahora una lista de acciones y construye un botón por cada una, sin conocer cuántas son ni qué hacen.

Dónde verlo. TurnoFiltroDialogos.crearAcciones, el constructor de TurnoFiltrosPanel y TurnoPanel.crearPanelInferior. TurnoPanel pasó de 291 a 208 líneas y ya no contiene diálogos de entrada ni conversiones de fechas. Un filtro nuevo se incorpora agregando una acción a la lista, sin modificar el panel de filtros.

P-03 – La gestión de turnos mezclaba consultas, comandos y facturación

Situación previa. TurnoConsultaService resolvía las búsquedas de turnos por distintos criterios y, además, calculaba el monto a facturar mediante un atributo Facturador que solo utilizaba el método calcularMonto. A su vez, la interfaz TurnoService reunía en un único contrato trece operaciones: alta, modificación, cambio de estado, baja, ocho consultas y el cálculo del monto. Cualquier cliente que necesitara solo una de esas capacidades quedaba obligado a depender del contrato completo.

Principios aplicados. Por Responsabilidad Única, las consultas cambian cuando aparecen nuevos criterios de búsqueda y la facturación cambia cuando se modifica la política de cobro, por lo que no deben convivir en la misma clase. Por Segregación de Interfaces, ningún cliente debería verse forzado a depender de operaciones que no utiliza.

Cambio aplicado. Se creó FacturacionTurnoService, que localiza el turno mediante TurnoResolutor y delega el cálculo en Facturador, y se retiró Facturador de TurnoConsultaService. El contrato de servicio se dividió en GestionTurnos, ConsultaTurnos y FacturacionTurnos. TurnoService se conserva como la unión de los tres, para que TurnoController mantenga una única entrada hacia el módulo.

Dónde verlo. Las interfaces GestionTurnos, ConsultaTurnos y FacturacionTurnos, la declaración de TurnoService, FacturacionTurnoService y el constructor de TurnoConsultaService, que dejó de recibir el facturador.

P-04 – MainFrame mezclaba composición visual con cierre y guardado

Situación previa. MainFrame construía la ventana y sus pestañas y, además, controlaba la política de cierre de la aplicación: registraba el WindowListener, preguntaba si se debían guardar los datos, ejecutaba el guardado y cerraba la ventana. La iteración anterior había logrado que MainFrame recibiera el guardado como un Runnable, sin conocer la persistencia, pero la decisión completa del flujo de cierre seguía dentro de la clase visual.

Principios aplicados. Por Responsabilidad Única, la composición y la navegación de la interfaz cambian por motivos visuales, mientras que la política de cierre y guardado cambia por decisiones sobre el ciclo de vida de la aplicación.

Cambio aplicado. Se creó ManejadorCierreAplicacion, que recibe la ventana y la acción de guardado, y se trasladaron a esa clase el WindowListener, la confirmación de salida y la ejecución del guardado. Main construye MainFrame y luego configura el manejador sobre ella.

Dónde verlo. Main.main, ManejadorCierreAplicacion.configurar y el constructor de MainFrame, que ya no recibe la acción de guardado. MainFrame pasó de 88 a 62 líneas y no contiene ninguna referencia a JOptionPane ni a eventos de ventana.

P-05 – Agregar una especialidad obligaba a modificar OdontologoFactory

Situación previa. Incorporar una nueva especialidad no se resolvía creando solamente su subclase de Odontologo. También había que agregar un caso en los dos switch de OdontologoFactory, uno para crear y otro para rehidratar, y ampliar el enum EspecialidadOdontologica. La jerarquía de odontólogos ya permitía el uso polimórfico, pero el mecanismo que la construía no lo hacía.

Principios aplicados. Por Abierto/Cerrado, el sistema debe poder ampliarse agregando código nuevo, sin modificar el que ya funciona. Cada especialidad nueva sumaba condicionales duplicados en una clase existente.

Cambio aplicado. Se creó la interfaz CreadorOdontologo y una implementación por especialidad: CreadorOdontologoGeneral, CreadorOrtodoncista y CreadorEndodoncista. OdontologoFactory dejó de ser una clase estática: ahora recibe en su constructor la lista de creadores, los registra por el código de su especialidad y resuelve tanto la creación como la rehidratación buscando el creador correspondiente. EspecialidadOdontologica pasó de enum a una clase con código y descripción, de modo que pueden definirse especialidades nuevas. La fábrica se inyecta en OdontologoServiceImpl y en la persistencia, y también provee la lista de especialidades que muestra el combo de la vista.

Dónde verlo. CreadorOdontologo y sus tres implementaciones, OdontologoFactory y la construcción de la fábrica en DependenciasClinica. OdontologoFactory ya no contiene ningún switch. Una especialidad nueva se incorpora creando su subclase y su creador, y registrando este último en DependenciasClinica.

P-06 – Las coberturas estaban cerradas dentro de un enum

Situación previa. Las políticas de cálculo de la cobertura estaban implementadas como constantes del enum CoberturaPaciente: PARTICULAR devolvía la tarifa base del odontólogo y OBRA_SOCIAL el copago fijo. Agregar una prepaga o un plan con porcentaje obligaba a editar el enum que reunía las políticas existentes. El resto del diseño ya era correcto: Paciente delegaba el cálculo en su cobertura, y ni Turno ni Facturador tenían condicionales según el tipo.

Principios aplicados. Por Abierto/Cerrado, la cobertura es un punto de variación del negocio y debe poder ampliarse con una implementación nueva e independiente, sin modificar las existentes.

Cambio aplicado. CoberturaPaciente pasó a ser una interfaz con las operaciones getCodigo, getDescripcion y calcularMonto, y se crearon CoberturaParticular y CoberturaObraSocial. Estas dos clases habían sido eliminadas en la primera iteración al adoptar el enum, que resolvía la duplicación pero cerraba la familia a nuevas variantes; ahora se recuperaron como implementaciones de la interfaz. Se creó además CatalogoCoberturas, que registra las coberturas disponibles, las entrega a la vista para el combo y resuelve por código o por descripción los valores leídos del archivo. La compatibilidad con los archivos anteriores se mantiene: la carga sigue aceptando true y false, y ahora los traduce mediante el catálogo.

Dónde verlo. CoberturaPaciente, sus dos implementaciones, CatalogoCoberturas y la construcción del catálogo en DependenciasClinica. Paciente, Turno y Facturador conservan su lógica sin cambios y continúan delegando el cálculo; una cobertura nueva se incorpora implementando la interfaz y registrándola en el catálogo.

P-07 – Los clientes recibían interfaces de repositorio demasiado amplias

Situación previa. ITurnoRepository heredaba de IRepository todas las operaciones de alta, baja, modificación y consulta, y les sumaba las consultas específicas de turnos y las dos verificaciones de conflicto horario. Ese contrato completo se entregaba a todos sus clientes, aunque cada uno usaba una parte mínima: DisponibilidadTurnos solo verificaba conflictos, TurnoConsultaService solo leía, TurnoServiceImpl solo escribía y TurnoResolutor solo buscaba por identificador. Este último, además, recibía las interfaces completas de los repositorios de pacientes, odontólogos y secretarias para usar una sola operación de cada una.

Principios aplicados. Por Segregación de Interfaces, cada cliente conocía métodos ajenos a su necesidad, y cualquier implementación alternativa habría tenido que ofrecer operaciones que no correspondían a su finalidad. Por Inversión de Dependencias, aunque las abstracciones ya existían, no representaban lo que cada módulo de alto nivel realmente necesitaba; depender de contratos mínimos reduce el acoplamiento con capacidades irrelevantes.

Cambio aplicado. Se dividió el contrato genérico en BuscadorPorId, RepositorioLectura y RepositorioEscritura, e IRepository quedó como la unión de lectura y escritura. Para los turnos se crearon ConsultaTurnosRepository, EscrituraTurnosRepository y VerificadorConflictosTurno, que ITurnoRepository combina. TurnoRepository implementa el contrato completo, pero cada colaborador recibe únicamente la capacidad que utiliza.

Dónde verlo. Los constructores de TurnoResolutor, que recibe cuatro BuscadorPorId; DisponibilidadTurnos, que recibe un VerificadorConflictosTurno; TurnoConsultaService, que recibe un ConsultaTurnosRepository; y TurnoServiceImpl, que recibe un EscrituraTurnosRepository.

P-08 – La persistencia dependía directamente de ArchivoTexto

Situación previa. Las cuatro clases de persistencia declaraban un atributo de tipo ArchivoTexto, y PersistenciaServicio creaba esa instancia en su propio constructor. Las reglas para convertir cada entidad en una línea de texto y reconstruirla quedaban así atadas al mecanismo físico que utiliza java.nio para leer y escribir archivos. FormatoLinea ya separaba el formato de los campos, pero la lectura y la escritura seguían declaradas contra la clase concreta.

Principios aplicados. Por Inversión de Dependencias, la política de persistencia es un módulo de alto nivel y no debe depender de un detalle de infraestructura. Cambiar a un almacenamiento en memoria, a otra codificación o a otro medio exigía modificar las clases de conversión, en lugar de sustituir una implementación.

Cambio aplicado. Se creó la interfaz AlmacenamientoLineas, con las operaciones leerLineas y escribirLineas, y ArchivoTexto pasó a implementarla. Las clases de persistencia reciben la abstracción por constructor, y PersistenciaServicio la recibe ya construida junto con el catálogo de coberturas y la fábrica de odontólogos. DependenciasClinica es quien crea la implementación concreta y la inyecta.

Dónde verlo. AlmacenamientoLineas, ArchivoTexto, los constructores de las clases de persistencia y DependenciasClinica. Una búsqueda sobre el código confirma que ArchivoTexto es la única clase de la capa de persistencia que importa java.nio, y que fuera de ella solo DependenciasClinica la menciona.

P-09 – La persistencia dependía de repositorios concretos

Situación previa. PersistenciaServicio y las clases de persistencia recibían y devolvían PacienteRepository, OdontologoRepository, SecretariaRepository y TurnoRepository, y durante la carga creaban internamente instancias concretas de esos repositorios. Los servicios ya dependían de interfaces desde la iteración anterior, pero la inversión de dependencias se interrumpía al llegar a la persistencia, y DependenciasClinica debía conservar los tipos concretos porque esas clases los exigían o los devolvían.

Principios aplicados. Por Inversión de Dependencias, la persistencia quedaba acoplada a implementaciones basadas en HashMap, y otra implementación de repositorio no podía participar del mismo flujo sin cambiar firmas y construcciones internas.

Cambio aplicado. El guardado recibe ahora RepositorioLectura y la carga recibe RepositorioEscritura, además de BuscadorPorId en el caso de los turnos, que necesitan resolver a sus actores. Los métodos de carga dejaron de crear y devolver repositorios: DependenciasClinica crea los repositorios concretos, se los entrega a la persistencia para que los complete y luego los inyecta en los servicios.

Dónde verlo. Las firmas de guardar, cargarPacientes, cargarOdontologos, cargarSecretarias y cargarTurnos en PersistenciaServicio, y el constructor de DependenciasClinica. Una búsqueda sobre el código confirma que la capa de persistencia ya no menciona ninguna clase de repositorio concreta.

5. Principio de Sustitución de Liskov

No se detectó una violación del Principio de Sustitución de Liskov, por lo que no se realizaron cambios específicos para este principio. Las subclases de Odontologo implementan getTarifaBase y puedeAtender con un comportamiento coherente con el contrato de la superclase, y cualquiera de ellas puede usarse donde se espera un Odontologo. Las subclases de Persona preservan las invariantes comunes, que se validan en el constructor de la superclase. Las excepciones específicas del sistema pueden sustituirse por ClinicaException, y ningún repositorio implementa operaciones vacías ni lanza excepciones por métodos no soportados.

Los cambios de esta iteración, además, refuerzan el cumplimiento del principio: las coberturas, los creadores de odontólogos y el almacenamiento se utilizan siempre a través de sus interfaces, de modo que cualquier implementación puede reemplazar a otra sin que los clientes lo noten. El único ajuste derivado se hizo en OdontologoServiceImpl, donde la comparación de especialidades pasó de != a equals, ya que EspecialidadOdontologica dejó de ser un enum.